

SemEval-2025 Task 7 — English Monolingual Fact-Check Retrieval

Full Pipeline Report: Methodology, Results, and Analysis

Track: Monolingual (eng → eng) **Task:** Given a social media post, retrieve the correct previously-fact-checked claim from a pool of candidate claims. **Base model:** intfloat/multilingual-e5-large (ME5-Large), a multilingual bi-encoder embedding model.

1. Task and Data Setup

The dataset comes from the official SemEval-2025 Task 7 release (“Multilingual and Crosslingual Fact-Checked Claim Retrieval”). This work uses only the English monolingual subtask: posts written in English are matched against a pool of English fact-checked claims.

Verified data counts (all confirmed against the paper’s Table 2 before any modeling began):

Split	Count	Verified against paper?
Fact-check pool (train/dev)	85,734	✓ exact match
Posts (train)	4,351	✓ exact match
Posts (dev)	478	✓ exact match
Train pairs (post ↔ claim)	5,446	✓ exact match
Dev pairs	627	✓ exact match
Test fact-check pool	145,287	✓ exact match
Test posts	500	✓ exact match
Test pairs	740	⚠ paper reports 574 — documented, unresolved discrepancy

The test-pair discrepancy (740 downloaded vs. 574 reported in the paper) was checked against two independent sources and does not affect the post/claim pool sizes, which match exactly. It is treated as a documented dataset-version difference rather than an error in the pipeline.

Text preprocessing: OCR fusion

Many posts have short, vague captions where the actual claim is only legible in an attached image (the paper notes this applies to roughly 15% of connections). To recover that signal, post text and OCR text were fused:

- If only one of {caption, OCR} exists, use it.
- If both exist and OCR is already contained in the caption, use the caption alone (avoids duplication).
- If both exist and differ, concatenate them.

Note on a formatting drift between sessions: the original notebook joined the two with an explicit tag ("`{caption} [OCR]: {ocr}`"), while the resumed notebook joined them with a plain space ("`{caption} {ocr}`"). This is a small, easy-to-miss inconsistency — it does not change the underlying idea, but it does mean the exact embeddings produced in the resumed session are not byte-identical to the original session's, which is why the two sessions' "sanity check" baseline numbers differ very slightly (see Section 4). For a clean write-up, the fusion function should be kept identical across all notebooks/sessions.

2. Stage 1 — Frozen Baseline

The un-fine-tuned ME5-Large model was used directly, with the model's required prefixes ("query: " for posts, "passage: " for claims), mean pooling over token embeddings, and L2 normalization, then cosine similarity (dot product of normalized vectors) between every post and every candidate claim, taking the top-10 by similarity.

Result (test set, first session):

Metric	Score
Success@1	0.434
Success@5	0.716
Success@10	0.788

This is the reference point every later stage is measured against. For comparison, the paper's own reported E5-Large baseline gets Success@10 = 0.818 — slightly higher than this reproduction, most plausibly due to differences in the OCR-fusion preprocessing, the specific test-pair subset (740 vs. 574), or minor implementation details (fp16 inference, exact truncation length) rather than any error in the pipeline.

3. Stage 2 — LoRA Fine-Tuning

3.1 Hard negative mining

Before fine-tuning, the frozen model was used to encode all claims and all training posts, then for each post the top-20 most similar claims were retrieved and the ones that were *not* the true positive were kept as "hard negatives" (5 per post). These are much more useful for contrastive training than random negatives, because they force the model to learn fine-grained distinctions between claims that already look similar in embedding space, rather than trivially distinguishing unrelated topics.

3.2 QLoRA configuration

- 4-bit quantization (NF4, double quant) via bitsandbytes
- LoRA rank $r=8$, $\alpha=16$, applied **only** to attention Q/K/V projections (explicitly excluding the feed-forward "dense" layers — an earlier version of this project had accidentally matched those too via substring matching, which over-adapted the embedding space on only 5.4k examples; narrowing the target modules was a deliberate fix)
- Dropout 0.1, bias untouched

3.3 Training objective

A multi-negative InfoNCE-style contrastive loss: - Each batch has `batch_size=16` queries, each with 1 positive and 3 hard negatives (`N_NEGS_PER_STEP=3`) - All positives *and* all negatives in the batch are concatenated into one candidate matrix - Cross-entropy loss over $(\text{query} \cdot \text{candidates}) / \text{temperature}$, with temperature = 0.05

Because every batch's positives are pooled together, this setup gets **in-batch negatives for free** in addition to the mined hard negatives — every other query's positive in the batch also acts as a negative for a given query. This is a theoretically sound and fairly strong contrastive setup, not a naive triplet loss.

- 3 epochs, effective batch size 32 ($16 \times \text{grad-accum } 2$), LR $3e-5$, linear warmup+decay
- Model selection: at the end of each epoch, the model was evaluated on the **dev set** (478 posts) using a weighted combination of Success@1/@5/@10 (weights 0.4/0.3/0.3), and only checkpoints that improved this combined score were kept as the “best” adapter — this prevents accidentally shipping a checkpoint that overfits to one metric.

3.4 Training curve

Epoch	Avg loss	Dev S@1	Dev S@5	Dev S@10	Combined score
1	2.278	0.360	0.730	0.803	0.604
2	1.346	0.473	0.764	0.820	0.664
3	1.248	0.479	0.762	0.829	0.669

Loss decreases smoothly and dev scores improve monotonically across all three epochs — no sign of overfitting within this short training run.

3.5 Result on test set

Metric	Frozen	Fine-tuned	Δ
Success@1	0.434	0.418	−0.016 (−3.7%)
Success@5	0.716	0.728	+0.012 (+1.7%)
Success@10	0.788	0.810	+0.022 (+2.8%)

Why Success@1 went down while Success@5/@10 went up — this is the single most important pattern in the whole project, and it repeats at every later stage.

Contrastive fine-tuning with hard negatives optimizes the model to pull the true positive *closer than a set of specific confusable negatives*, not to guarantee it lands in the single best-ranked slot against the *entire* 145k-claim pool. With only ~5,400 training examples and 3 epochs, the fine-tuning nudges the embedding space just enough to fix cases where the correct claim was previously ranked, say, 6th–10th (which directly helps Success@5/@10), but in doing so it can also occasionally swap the order of the top 1–2 candidates for posts that the frozen model was already getting exactly right — a small number of “already correct” cases regress to 2nd place. Because Success@1 is the strictest, least forgiving metric (only the single top slot counts), it is the most sensitive to this kind of embedding-space reshuffling, while Success@5/@10 are much more tolerant of it. This is a known and reported trade-off in embedding fine-tuning literature, not a bug in this implementation.

4. Stage 3 — Ensemble ($\alpha = 0.65$)

Motivation: the frozen model is better at Success@1; the fine-tuned model is better at Success@5/@10. Averaging their similarity scores lets you keep both strengths.

Method: for a tunable weight α , compute `combined_similarity =  $\alpha \cdot \text{sim\_finetuned} + (1 - \alpha) \cdot \text{sim\_frozen}$` , re-rank by this combined score. α was swept on the **dev set only** (never on test, to avoid leakage), with the specific objective: *maximize Success@1, subject to Success@5 and Success@10 not decreasing relative to the fine-tuned-only baseline*. This constrained search selected $\alpha = 0.65$.

Test result:

Metric	Frozen	Fine-tuned	Ensemble ($\alpha=0.65$)
Success@1	0.434	0.418	0.440
Success@5	0.716	0.728	0.750
Success@10	0.788	0.810	0.824

This is the first stage where **every metric improves over the frozen baseline simultaneously** — the ensemble genuinely resolves the Success@1 regression from Stage 2 rather than just averaging it away.

5. Stage 4 — Re-Tuned Ensemble ($\alpha = 0.20$)

In a later session, the dev-set-encoded embeddings were recomputed and α was swept again — **but with a different, simpler objective this time: directly maximize Success@10 alone**, with no constraint protecting Success@1 or Success@5. This is a genuinely different selection rule, not a re-run of the same search, which is why it landed on a different value.

Why $\alpha = 0.20$ (mostly frozen, only 20% fine-tuned) instead of 0.65: when the search is only asked to maximize Success@10, it naturally gravitates toward whichever mixture ranks the correct claim *somewhere* in the top 10 most often — and since the frozen model already had strong raw Success@1 (0.434) and reasonable Success@10 (0.788), weighting it more heavily keeps that strength intact while the smaller 20% contribution from the fine-tuned model still nudges a meaningful number of borderline cases (where the correct claim was ranked 11th-20th under frozen-only scoring) into the top 10.

Test result:

Metric	Ensemble ($\alpha=0.65$)	Ensemble ($\alpha=0.20$)
Success@1	0.440	0.476
Success@5	0.750	0.766
Success@10	0.824	0.834

Notably, this version beats the $\alpha=0.65$ ensemble on **all three metrics**, including Success@1 — which the $\alpha=0.65$ search was explicitly optimizing for and this one wasn't. This should be read as: the $\alpha=0.65$ constrained search found a *locally* good trade-off under its own rule, but a different, unconstrained rule happened to find a mixture that is *globally* better on this particular test set. It does not mean the first search was wrong — it means the two searches were

solving different optimization problems on the same small dev set, and results on a 478-example dev set can be somewhat sensitive to exactly which objective is used (see Section 8 for a caveat on this).

6. Stage 5 — Retrieve-then-Rerank with a Cross-Encoder

6.1 Justifying the extra stage: the “headroom” check

Before adding a reranker, the pipeline checked whether there was anything to gain: on the dev set, Success@10 (top-10 retrieval) was 0.845, but Success@50 (is the correct claim anywhere in the top 50?) was 0.906 — a **6-point headroom**. This means for roughly 6% of dev posts, the bi-encoder ensemble already has the correct claim sitting somewhere between rank 11 and rank 50; it’s a ranking problem, not a recall problem, at that point. This is exactly the situation a second-stage reranker is designed to fix, and checking for headroom before building one is good practice — it confirms the extra complexity is justified rather than assumed.

6.2 Reranking mechanism

- Retrieve the top-50 candidates per post using the tuned ensemble ($\alpha=0.20$) bi-encoder scores.
- Score each (post, candidate) pair with a cross-encoder, BAAI/bge-reranker-v2-m3, which reads the post and claim text *together* (unlike a bi-encoder, which scores them independently) and can therefore capture much finer-grained relevance signals.
- Fuse the cross-encoder score with the candidate’s original bi-encoder rank position using a tunable weight β : $\text{fused} = \beta \cdot \text{CE_score} + (1 - \beta) \cdot \text{rank_position_score}$. $\beta=0$ reduces to pure bi-encoder ranking (used as a sanity check — it exactly reproduced the ensemble’s own dev score, confirming the fusion code is implemented correctly); $\beta=1$ is pure cross-encoder ranking.

6.3 Fine-tuning the cross-encoder

The zero-shot bge-reranker-v2-m3 was then fine-tuned on this task for 1 epoch using 5,446 positive (post, true claim) pairs and 4 **randomly sampled** negatives per positive drawn from the full 85k claim pool (27,230 training examples total).

This is the one place in the whole pipeline where the “use hard negatives” lesson from Stage 2 was not carried forward. Random negatives from an 85k pool are, almost by definition, about a completely unrelated topic — the cross-encoder mostly learns “is this even the same subject,” which is a much easier and less useful skill than what it actually needs at inference time: discriminating between the top-50 candidates that a strong bi-encoder has *already* judged to be topically similar. This doesn’t mean the fine-tuning was wasted (the results below show a real improvement), but it is very likely leaving performance on the table — reusing the same hard-negative-mining code from Stage 2, applied to the bi-encoder’s own top-50 output, would be the natural next experiment.

6.4 Beta sweep and final result

Beta was swept on dev in steps of 0.1; Success@10 rose roughly monotonically from 0.845 ($\beta=0$) to a peak of 0.858 at **$\beta=1.0$** — i.e., the best dev configuration ignores the original retriever ranking entirely and trusts the fine-tuned cross-encoder completely.

Test result:

Metric	Ensemble ($\alpha=0.20$, no rerank)	+ CE Rerank ($\beta=1.00$)
Success@1	0.476	0.440
Success@5	0.766	0.782
Success@10	0.834	0.860

The same pattern as every prior stage repeats: Success@5/@10 improve, Success@1 regresses — for the same underlying reason described in Section 3.5 (a reranking objective trained to distinguish confusable claims within a candidate set is not the same objective as “always put the single correct answer in first place,” and the two can trade off against each other, especially when the reranker’s training negatives don’t closely match its deployment-time candidate distribution).

7. Full Results Table — Every Stage

Stage	Success@1	Success@5	Success@10
Frozen ME5-Large	0.434	0.716	0.788
Fine-tuned ME5-Large (LoRA)	0.418	0.728	0.810
Ensemble ($\alpha=0.65$)	0.440	0.750	0.824
Ensemble ($\alpha=0.20$, re-tuned)	0.476	0.766	0.834
Ensemble + CE Rerank ($\beta=1.00$)	0.440	0.782	0.860

External reference points (from the paper):

System	Success@10
Paper’s own E5-Large baseline	0.818
This pipeline’s final result	0.860
Paper’s top system (PINGAN AI)	0.916

The final pipeline beats the paper’s own reported E5-Large baseline by 4.2 points and closes roughly 40% of the remaining gap to the top leaderboard system, using only bi-encoder fine-tuning, score-level ensembling, and a single cross-encoder rerank stage — no BM25, no multi-model ensembles, no translation-based augmentation.

8. Known Issues, Caveats, and Things to Verify Before Final Submission

1. **Results-saving crash (fixed, but verify the fix ran):** the cells that were meant to save the final comparison table and predictions for the reranked stage crashed with `NameError: name 'OUTPUT_DIR' is not defined`, almost certainly because the Colab runtime disconnected and `OUTPUT_DIR` (defined early in the notebook) was lost from memory when later cells were re-run without re-running the setup cell. **Confirm the re-saved table on Drive matches the numbers in this report before submitting**, since the original save attempt never completed.

2. **Cross-encoder trained on random, not hard, negatives** (Section 6.3) — the single highest-value follow-up experiment. Expect a further Success@10 gain, and possibly a smaller Success@1 regression, if the CE is retrained on hard negatives mined from its actual top-50 inference-time candidate pool.
 3. **Small dev set (478 posts) means hyperparameter searches carry real noise.** A rough calculation: the standard error on a proportion near 0.85 with $n=478$ is about ± 2 percentage points — which is roughly the same size as the *entire spread* between the best and worst beta values tested (0.845 to 0.858). The fact that the winning beta (1.0) sits at the very edge of the search range is a classic signal that the “optimum” may be partly an artifact of dev-set noise rather than a robustly better setting. This doesn’t invalidate the result, but it’s worth a caveat in any formal write-up, and ideally a bootstrap confidence interval before stating $\beta=1.0$ as definitively best.
 4. **An unresolved diagnostic anomaly.** A sanity check on the first 200 test posts found that the cross-encoder reranking changed **zero** posts’ top-1 pick and **zero** posts’ top-10 candidate set, yet the aggregate metrics over the full 500 posts did move. This means all of the measured improvement is concentrated in the untested remaining 300 posts — plausible, but unverified. Before treating the CE rerank’s contribution as fully trustworthy, this check should be re-run across all 500 test posts.
 5. **OCR-fusion formatting drifted slightly between sessions** (Section 1), causing the “sanity check” frozen/fine-tuned baseline numbers to differ by a few tenths of a point between the two notebooks. Not a correctness issue, but worth keeping identical for clean reproducibility going forward.
 6. **The test-pair count discrepancy (740 vs. paper’s 574)** remains unresolved but is well-documented and does not affect posts/claim-pool sizes.
-

9. On Additional Metrics (Precision/Recall/F1, BLEU, BERTScore)

For future work, it’s worth being precise about which additional metrics are actually meaningful for this task:

- **Precision@k / Recall@k / F1@k** are computable from the saved per-post ranked prediction lists and are legitimate to report, but since almost every post here has exactly one correct claim, Recall@k reduces to the same information as Success@k, and Precision@k is just $1/k$ or $0/k$ — so they add rigor/completeness to a report but little new insight.
 - **BLEU and BERTScore are not a good fit for this task.** Both compare a *generated* text against a *reference* text, but this pipeline retrieves an existing claim by ID rather than generating text — there is no generated string to score, and comparing the text of a correctly-retrieved claim to itself is trivial while comparing the text of an incorrectly-retrieved claim to the true one measures incidental wording overlap, not retrieval quality.
 - **MRR and NDCG@10** are the standard, well-fitted companions to Success@k for exactly this kind of ranked-retrieval task (they’re what real TREC/SemEval-style leaderboards report alongside Success@k), and the code for both already exists in the notebook but was never called on the saved predictions — this is the natural, low-effort way to strengthen the metrics section.
-

10. Summary of the Recurring Finding

The single most important empirical pattern across this entire project, worth stating explicitly in any conclusion: **every fine-tuning or reranking stage that improved Success@5/@10 also cost some Success@1**, and the ensemble stages are the exception precisely because they were explicitly built to protect against this trade-off by mixing in the frozen model's stronger top-1 behavior. This is not a series of unrelated ups and downs — it's the same underlying tension (optimizing for "correct answer somewhere in the top-k" vs. "correct answer specifically in first place") showing up at every stage where a model was fine-tuned with a contrastive or pairwise ranking objective.

11. Recommended Next Steps

1. Verify the results-saving fix actually wrote the final comparison table to Drive.
2. Retrain the cross-encoder using hard negatives mined from the ensemble's own top-50 output (highest expected payoff).
3. Widen the top-1/top-10-set diagnostic check to all 500 test posts, not just the first 200.
4. Compute MRR and NDCG@10 on the saved prediction files using the already-written functions.
5. Run a bootstrap or simple confidence-interval estimate on the dev set before finalizing the beta=1.0 choice.
6. Standardize the OCR-fusion function across all notebooks/sessions.